Offline Intelligent Lossless Compression
of Hyperlinked Documents

A Thesis

Presented to the

Faculty of

California State University, Fullerton

---

in Partial Fulfillment

of the Requirements for the Degree

Master of Science

in

Computer Science

By

Brian Edward Croner

Approved by:

Dr. Kevin Wortman, Committee Chair Department of Computer Science	Date
Dr. Bin Cong, Committee Member Department of Computer Science	Date
Dr. Xiong Wang, Committee Member Department of Computer Science	Date

TABLE OF CONTENTS

Chapter Page

1 Introduction . . . 1
1.1 Motivation . . . 1
1.2 Our Concept . . . 1
1.3 Target Document . . . 2

2 Previous Work . . . 4
2.1 Run-Length Coding . . . 4
2.2 Huffman . . . 4
2.3 LZ-Family . . . 4
2.4 Burrows-Wheeler Transform . . . 5
2.5 Arithmetic Coding . . . 5
2.6 Prediction by Partial Matching . . . 5
2.7 PAQ . . . 5
2.8 Grammar-Based Compression . . . 6

3 Definitions . . . 7

4 Methodology . . . 8
4.1 Stand-Alone Compression Tool . . . 8
4.1.1 First Phase . . . 8
4.1.2 Second Phase . . . 9
4.1.3 Implementation Details . . . 10
4.1.4 Compression Notes . . . 11
4.2 Stand-Alone Decompression Tool . . . 11
4.3 Preprocessor to Huffman . . . 12

5 Experimental Results . . . 13
5.1 Environments . . . 13
5.1.1 Local Environment . . . 13
5.1.2 Target Environment . . . 13
5.2 Huffman and Gzip Benchmarks . . . 13
5.3 Huffman Preprocessor on the Large Document . . . 14
5.4 Stand-Alone Compressor Tool on the Small Document . . . 15
5.5 Stand-Alone Compression Tool on the Large Document . . . 16

6 Conclusion . . . 17
6.1 Comparison of Approaches . . . 17

Chapter Page

6.2 Future Work 17

REFERENCES 19

LIST OF TABLES

Table Page

5.1 Huffman Preprocessor on Large Document 14
5.2 Stand-Alone Compressor on Small Document, Constant Cut Offs 15
5.3 Stand-Alone Compressor on Small Document, Calculated Cut Offs . . . 16
5.4 Stand-Alone Compressor on Large Document 16

List of Algorithms

ACKNOWLEDGMENT

I would like to thank all the LaTeX style files that exist on the Internet for allowing me to become a living file. In particular, style files from the following universities were referenced during the creation of the CSUF style file, University of California, Princeton University, Harvard University, Standford University and New Jersey Institute of Technology.

This is the second paragraph of my acknowledgement. Thank you and have a nice day.

This work is dedicated to
my mom and dad

CHAPTER 1

Introduction

1.1 Motivation

Lossless compression reduces storage requirements, reduces bandwidth requirements, and preserves the integrity of the files processed. Current state-of-the-art lossless text compression algorithms fall far short of Shannon's estimate for the compression of a 100 MB XML document [11].

One on hand, image, video, and audio data can be compressed with a lossy format without greatly compromising the user experience [15]. If sufficient lossless compression of the original raw data could be achieved, the user experience could be greatly enhanced, opening new markets for cinema-like experiences in a home entertainment system. On the other hand, one might imagine that loss of various data from the lossy compression of text may be considered unacceptable. If this data can be compressed more effectively, it would reduce the storage requirements for XML database backups and the transmission bandwidth requirements of XHTML documents.

1.2 Our Concept

We consider the concept of grammar-aware compression in a loose and highly fundamental sense, where a grammar is a translation from a single start nonterminal to the

string that was compressed via a progressive expansion of each nonterminal to expanded translations of those nonterminals. We borrow from common nomenclature the terms sequence and selection, based on the 1966 proof that all computable functions can be represented by the sequence, selection, and iteration operations [3] by Bhm and Jacopini. We represent a string using sequence transformation operations and selection transformation operations, where each operation performs a specific transformation using a two-part dictionary definition.

We define a compression tool that is designed to be competitive with state-of-the-art XML text compressors but is general enough to compress image, video, and audio files with some modifications. It will be highly valuable to the industry and may open up a new market in offering a better user experience to home entertainment consumers. Our algorithm is an iterative, two-phase compression tool which provides us with a compression ratio to be compared against benchmarks on a large XML document.

1.3 Target Document

For reference we first experiment with a variation of our algorithm where we use our first phase sequence transformations as a preprocessor to a Huffman compressor which has been modified to accommodate a document of a large alphabet of symbols. We then show that a configuration of our algorithm results in a better compression ratio than the best configuration of our preprocessed Huffman technique. In fact, our algorithm's best observed compression ratio is better than that of any run of

regular Huffman, gzip, and preprocessed Huffman that we observed.

The algorithm is run in four configurations on a popular 100,000,000-byte XML document, and the effectiveness of the approach is compared against industry benchmarks in pursuit of a popular international research prize, the Hutter Prize [11]. To test various configurations at a lesser investment of time, we also run our algorithm on a 105,310-byte HTML document [7].

CHAPTER 2

Previous Work

2.1 Run-Length Coding

Run-length coding is perhaps the simplest method of suppressing patterns of simple repetition. The character to be repeated is specified, and a number indicating the repetitions of that character is provided.

2.2 Huffman

Huffman coding implements a prefix tree whereby the most common characters are represented with the least numerous sequences of bits, and wherein no sequence is a prefix of another sequence [8].

2.3 LZ-Family

Lempel and Ziv published a 1977 article [17] that decompresses data and references into a window of this decompressed data to repeat sequences already decompressed. It can be argued that this is an abstraction of run-length encoding where instead of repeating a single character, a sequence of characters is repeated. In 1978 they introduced a follow-up where a dictionary is maintained [18] where each symbol represents a sequence of characters, so as to look ahead instead of look behind. In 1984, Welch provided a method to implement this dictionary look-ahead approach in a very simple manner [14] in what is commonly known as the LZW algorithm.

2.4 Burrows-Wheeler Transform

The Burrows-Wheeler Transform examines the permutations of this string. The string is rotated and sorted for each possible rotation [4], where the last character in the sort is the character used to represent this position. Such a transformation lends itself to run-length encoding, which is applied to the output sequence.

2.5 Arithmetic Coding

Arithmetic coding reduces a sequence of bits to k (hopefully) repeating symbols [13]. When a match for a symbol occurs, its index $(0 \ldots k-1)$ is output. By placing this series of indexes behind a decimal point, a number between 0 and 1 is formed, based k. A conversion of based algorithm is applied to form the equivalent number base 2, which is a string of bits.

2.6 Prediction by Partial Matching

By using recently decompressed data in a left-to-right approach, one may predict the next bit or byte by applying a model to this recently decompressed data [5, 12].

2.7 PAQ

PAQ is a state-of-the-art grammar-unaware compression utility currently holding the record compression of the first 100,000,000 bytes of Wikipedia, implementing various models aimed at predicting the next in a series of bits using prediction by partial matching [16].

2.8 Grammar-Based Compression

Grammar-based compression is the discovery of a context-free grammar that can be used to model patterns that can resolve to the given input string [9], involving a grammar transformation phase and a grammar coding phase. Discovering the minimum grammar has been shown to be an NP-Complete problem [2]. Intelligent text compression using grammar-based codes is an of research that hasn't developed enough to exceed the compression capabilities of the state-of-the-art grammar-unaware text compression utility holding the record for the Hutter Prize [11].

CHAPTER 3

Definitions

CHAPTER 4

Methodology

4.1 Stand-Alone Compression Tool

As stated, our compression algorithm is a two-phase iterative process that gradually reduces the length of the list of symbols representing our document to one symbol, our start symbol. We produce and process all files in human-readable form, as converting textual integers to a bitstream is trivial.

After a simple conversion of a list of unsigned byte values into a symbol list of integers having the same values, our algorithm begins its first iteration. In the first phase of an iteration, we reduce the length of the symbol list by identifying and applying a series of sequence transformations to our symbols. In the second phase of an iteration, we reduce the size of the alphabet of symbols occurring in the symbol list by identifying and applying a series of selection transformations to our symbols. Iterative applications of our two-phase algorithm eventually reduce the length of the symbol list down to one symbol, our start symbol. At this point the algorithm has finished processing the document. The output of the compression tool is a syntax of first-phase transformations, second-phase transformations, and code data to guide the decompression of our symbol table into the original document.

4.1.1 First Phase

The algorithm's task during the first phase of an iteration is to reduce the length of our symbol list by identifying and applying new sequence transformations to our

symbols. A sequence transformation replaces every sequence of the two existing symbols y and z occurring in a sequence yz with an instance of the single new nonterminal w, while placing the definition $w \rightarrow yz$ into a dictionary for the first-phase syntax.

To locate the highest-frequency two-symbol sequence we sort the indices of all sequences of two symbols in nondecreasing order. We then count the number of indices that match the often most occurring two symbol sequence. In cases where there is more than 1 sequence with the same highest frequency value, we accumulate all possible sequences of two symbols occurring with this frequency.

We repetitively search for and apply high-frequency two-symbol sequences until a configurable cutoff is reached. At this point we record to our code data the number of sequence transformations occurring in the phase and switch to the second phase of our iteration with our symbol list having been reduced in size. In our symbol list we represent sequence symbols as positive integers that begin at 256 and increment for every successive sequence nonterminal symbol created.

4.1.2 Second Phase

The algorithm's task during the second phase of an iteration is to reduce the size of the alphabet of symbols occurring in our symbol list by identifying and applying new selection transformations to our symbols. A selection transformation replaces every instance of two existing nonterminals y and z with an instance of the new nonterminal w, while at the same time recording their identities onto the selection

operation bit stack (where y may be denoted with a 0 and z may be denoted with a 1).

To locate the two symbols with the lowest nonzero frequency we build a histogram of all symbols that occur in the symbol list. We then sort the indices of the histogram in nondecreasing order to give us a list of symbols ordered by frequency. We then select the two symbols that have the minimum nonzero frequency. In cases where there are more than two symbols with that same minimum nonzero frequency, we accumulate as many pairs of these symbols as possible before building a new histogram to sort.

We repetitively search for and apply low-frequency pairs of symbols until a configurable cutoff is reached. At this point we record to our code data the number of selection transformations occurring in this phase and proceed to the next iteration of our algorithm. In our symbol list we represent selection symbols as negative integers that begin at -1 and decrement for every successive selection nonterminal symbol created.

4.1.3 Implementation Details

The sequence and selection symbol tables are each stored as a list of ordered pairs of integers. Our code data gives us the number of symbols created in each phase, and from it we can find the total number of each type of nonterminal. To represent selection transformation symbols (negative integers) in our compressed syntax file we find the highest-order sequence transformation symbol and subtract from that value the negative number representing the selection transformation symbol. We can then merge the two symbol tables into one combined two-part table of pairs of

integers. Header information gives us the size in bits of our disambiguation bit stack and number of integers that represent the phases our code data contains.

4.1.4 Compression Notes

After iterative applications of our sequence and selection transformation phases we are left with a symbol list of an alphabet of size 1. At this point we proceed to the final sequence transformation phase, in which we apply sequence transformations until we are left with a single symbol. This sequence transformation symbol is our start symbol that we begin with when decompressing the syntax to reproduce the original document.

We experimented with various criteria to determine how many nonterminal rules should be generated in each phase. In general we reduced the size of the alphabet present in the symbol list to one-half the size it was prior to the iteration (i.e., we went from 256 symbols to $128, 64, \ldots, 1$). In a way the selection transformations phase balanced the sequence transformations phase, which increased the size of the alphabet present in our symbol list.

4.2 Stand-Alone Decompression Tool

To decompress the syntax file we use the header data to split it into code data, a sequence nonterminal symbol table, a selection nonterminal symbol table, and a disambiguation bit stack. We then transform the selection symbols back into their original negative number representations by subtracting any index greater than our highest-order sequence nonterminal symbol from this highest-order value. All data

input to our tool is, again, in human-readable form.

We instantiate a linked list with this highest-order sequence nonterminal value because this is our start nonterminal. Using a separate index for both of our symbol tables, one for sequence transformations and one for selection transformations, we repetitively apply our symbols in the reverse order that they were generated to gradually build our original list of data. The indices start at the highest-order entries and decrement after each application of a nonterminal rule to give us the next rule to apply.

After iteratively applying the nonterminals in reverse order, we are left with a linked list of symbols that all fall within the range $[0 \ldots 255]$. We convert these integers to unsigned characters and pass them in order to the standard output.

4.3 Preprocessor to Huffman

We applied the sequence transformations alone in a single phase and output the resulting symbol list to a modified Huffman compression algorithm for a reference to compare against. We experimented with different symbol table sizes, for our purposes allocating exactly as many symbols as would be necessary to fully utilize a given number of bits per symbol. For instance, for 19-bit symbols, we had $2^{19} - 256$ symbol table entries of sequence transformation production rules. We modified a Huffman algorithm implementation [6] to produce the canonical Huffman coding for the resulting symbol list and measured the result. After running the modified Huffman decompressor we decompressed the symbol list to yield the original document.

CHAPTER 5

Experimental Results

5.1 Environments

5.1.1 Local Environment

Compression and decompression tools were implemented on a laptop PC running Knoppix Linux LiveDVD [10] using 2 GB memory, a 2 GHz Intel Core 2 Duo CPU and over 50 GB free HD space. The compression and decompression tools were implemented in C to conserve space. When decoding Huffman coding to our preprocessed symbols, a heavy system load was at force because we were running several Huffman decoding processes simultaneously.

5.1.2 Target Environment

The target machine for the competition, by comparison, runs on Linux using 1 GB memory, a 2 GHz P4, 10 GB of HD space, and only 10 hours time to decompress the document.

5.2 Huffman and Gzip Benchmarks

Our large 100,000,000-byte target document is provided pre-compressed by the gzip algorithm [1] at a compressed size of 35,012,219 bytes. Running the canonical Huffman algorithm [6] on the large target document gives a compressed size of 63,861,987 bytes.

Table 5.1 Huffman Preprocessor on Large Document

bits	symbols	size	Huffman-coded Symbols	Huffman decode time	combined size
9	256	576	53753022	3m 23s	53753598
10	768	1920	48927287	8m 1s	48929207
11	1792	4928	45115685	15m 25s	45120613
12	3840	11520	41758881	27m 57s	41770401
13	7936	25792	38775139	49m 12s	38800931
14	16128	56448	36110612	85m 46s	36167060
15	32512	121920	33798232	149m 1s	33920152
16	65280	261120	31803575	259m 55s	32064695
17	130816	555968	29980198	455m 44s	30536166
18	261888	1178496	28321309	804m 36s	29499805
19	524032	2489152	26675621	1426m 49s	29164773
20	1048320	5241600	24740266	2532m 36s	29981866

5.3 Huffman Preprocessor on the Large Document

We tested a range of bit sizes to represent the symbol space, occupying as much of the entire symbol space as possible given the symbol bit size. For instance, given 10 bits, we create 768 symbols because, although 1024 total symbols are possible, the first 256 symbols are implied and must be reserved for our terminal symbols (the decompressed original document characters). Our best compression ratio was observed when we used 19 bits per symbol, which, when combined with the canonical Huffman technique, gave a final compressed size of 29164773 bytes (excluding the size of the executable utilities).

Table 5.2 Stand-Alone Compressor on Small Document, Constant Cut Offs

Sequence Cut Off	Sequence Symbols	Selection Symbols	D Bits	Final Size
2	8426	6210	106150	38954
3	5789	5073	151794	38055
4	4594	4168	173328	37072
5	3831	3547	189062	35694
6	3314	3115	199036	35399
7	2984	2893	210571	35911

5.4 Stand-Alone Compressor Tool on the Small Document

We tested various settings on the compression algorithm for the cutoff of the sequence transformation phases on our 103-Kb test document while keeping the selection transformation set to cut off after the size of the alphabet in the symbol list is one half what it was prior to the beginning of the iteration. Let $|L|$ represent the length of the symbol list at any point in time. Note that a sequence transformation phase cutoff value represents the minimum number of occurrences of two symbols y and z that appear in a sequence yz that in the document. If at least one sequence exists with at least this minimum frequency, this sequence is recorded in the sequence symbol table and the corresponding transformation (to w, as $w \rightarrow yz$) is applied. If no such sequences exist with at least this minimum frequency, the sequence transformation phase completes and the iteration continues to commence the selection transformation phase.

We also experimented with altering the selection transformation phase cutoff settings. Let $|E'|$ be the size of the alphabet of symbols occurring in the symbol list prior to beginning a selection transformation phase. Note that a selection transformation phase cutoff value represents the size of an alphabet in the current symbol

Table 5.3 Stand-Alone Compressor on Small Document, Calculated Cut Offs

Sequence Cut Off	Selection Cut Off	Sequence Symbols	Selection Symbols	D Bits	Code Numbers	Final Size
$log_2(\|L\|)$	$2 * \|E'\|/3$	2501	2490	258844	196	46738
$log_2(\|L\|)$	$1 * \|E'\|/3$	2218	2205	261462	60	40111
$log_2(\|L\|)$	$1 * \|E'\|/2$	2369	2356	259565	102	40532

Table 5.4 Stand-Alone Compressor on Large Document

Sequence Cut Off	Sequence Symbols	Selection Symbols	D Bits	Time	Final Size
$log_2(\|L\|)$	154938	154054	233421493	6512m	29911615
2	2082102	1902554	174536546	12100m	32774945
6	540295	531242	208139522	7965m	28830297

list. If The size of the alphabet is greater than this parameter, the algorithm continues to find pairs of symbols with a minimum frequency. If the size of the alphabet equals this parameter, the selection phase terminates and a new iteration of the algorithm iteration begins.

5.5 Stand-Alone Compression Tool on the Large Document

We tested four settings of our compression algorithm on the 100,000,000-byte document. We let $|L|$ represent the length of the symbol list at any point in time. We chose to cut off the selection transformation phases when the measured size of the alphabet in the symbol list is equal to one half the size of the alphabet prior to the beginning of the iteration of the algorithm in which the selection phase occurs.

CHAPTER 6

Conclusion

6.1 Comparison of Approaches

All final compression ratios using our sequence transformations as a preprocessor to the canonical Huffman algorithm were better than running the same canonical Huffman algorithm on the document without any preprocessing, and some configurations of our Huffman preprocessor yielded a significantly better compression ratio than that of gzip. We observed the greatest compression ratio with one configuration of our stand-alone algorithm, where with a simple constant as the cutoff parameter we were able to achieve a better compression ratio than any of the other approaches we considered. It must be noted that a significant investment of time is required to achieve the savings in space.

6.2 Future Work

We feel a lot of improvement can be made to the compression ratio of our algorithm if one could find more appropriate phase cutoff criteria. We witnessed an explosion of disambiguation bits with all our selection phase cutoffs, and we feel a significant savings in space can be achieved with a better balance between the space used for symbols and the space used for disambiguation bits.

The compression algorithm's running time can also be improved. By using a hash table to identify the most common sequence of two symbols instead of sorting

all the indices of the list only to find the most common one, we can reduce a major time cost of the algorithm from $O(nlgn)$ to $O(n)$.

In addition, we feel our algorithm can be adapted to compress image, video, and sound files. Decompressing our 103-Kb test file took seconds, and our algorithm was not fine-tuned to give a better compression ratio with any specific file.

REFERENCES

1. Gzip file format specification version 4.3, 1996. This is an electronic document. Date of publication: May 1, 1996. Date retrieved: March 19, 2012. Date last modified: [Date unavailable].

2. [Anonymous]. Lossy compression, 2011. This is an electronic document. Date of publication: [Date unavailable]. Date retrieved: April 30, 2011. Date last modified: [Date unavailable].

3. [Anonymous]. Paq, 2011. This is an electronic document. Date of publication: [Date unavailable]. Date retrieved: April 15, 2011. Date last modified: [Date unavailable].

4. Arpe, J., and Reischuk, R. On the complexity of optimal grammar-based compression. In *Data Compression Conference, 2006. DCC 2006. Proceedings* (march 2006), pp. 173 – 182.

5. Böhm, C., and Jacopini, G. Flow diagrams, turing machines and languages with only two formation rules. *Commun. ACM 9*, 5 (May 1966), 366–371.

6. Burrows, M., and Wheeler, D. J. A block-sorting lossless data compression algorithm.

7. Cleary, J., and Witten, I. Data compression using adaptive coding and partial string matching. *Communications, IEEE Transactions on 32*, 4 (apr 1984), 396 – 402.

8. Dipperstein, M. Huffman code discussion and implementation, 2008. This is an electronic document. Date of publication: [Date unavailable]. Date retrieved: March 19, 2012. Date last modified: [Date unavailable].

9. Doehla, S. Rtp payload format for mpeg-4 audio/visual streams, 2011. This is an electronic document. Date of publication: [Date unavailable]. Date retrieved: March 19, 2012. Date last modified: [Date unavailable].

10. Huffman, D. A method for the construction of minimum-redundancy codes. *Proceedings of the IRE 40*, 9 (sept. 1952), 1098 –1101.

11. Kieffer, J., and Yang, E.-H. Grammar-based codes: a new class of universal lossless source codes. *Information Theory, IEEE Transactions on 46*, 3 (may 2000), 737 –754.

12. Knopper, K. Knoppix - live linux filesystem on cd - knopper.net consulting, 2012. This is an electronic document. Date of publication: [Date unavailable]. Date retrieved: March 19, 2012. Date last modified: [Date unavailable].

13. Mahoney, M. €50,000 prize for compressing human knowledge, 2011. This is an electronic document. Date of publication: [Date unavailable]. Date retrieved: April 25, 2011. Date last modified: [Date unavailable].

14. Moffat, A. Implementing the ppm data compression scheme. *Communications, IEEE Transactions on 38*, 11 (nov 1990), 1917 –1921.

15. Rissanen, J., and Langdon, G. G. Arithmetic coding. *IBM Journal of Research and Development 23*, 2 (march 1979), 149 –162.

16. T.A., W. A technique for high performance data compression. *IEEE Comput. 17* (1984), 8–19.

17. Ziv, J., and Lempel, A. A universal algorithm for sequential data compression. *Information Theory, IEEE Transactions on 23*, 3 (may 1977), 337 – 343.

18. Ziv, J., and Lempel, A. Compression of individual sequences via variable-rate coding. *Information Theory, IEEE Transactions on 24*, 5 (sep 1978), 530 – 536.